

Keywords: test-time compute • search policy • reinforcement learning • LLM reasoning

Beyond Repeated Sampling

Learning Search Policies for LLM Reasoning via Reinforcement-Trained Concept Generation

By Ismail Labiad, Matthieu Kowalski, Marc Schoenauer, Rémi Munos & Julia Kempe (Meta FAIR)

arXiv:2609.26704 • September 2026

A small concept generator, trained with reinforcement learning to maximize the pass@k of a frozen large answer generator, achieves substantially higher diversity and downstream accuracy than naive repeated sampling at the same compute budget—and transfers to answer generators it was never trained against, including those from a different model family.

AT A GLANCE

Problem: Naive repeated sampling for test-time compute explores only through local decoding noise, producing near-duplicate attempts rather than semantically diverse solutions.

Why it matters: Hard reasoning problems require qualitatively different strategies; pass@k saturates quickly when all samples share the same implicit approach.

Method: Train a small concept generator with RL to emit problem-specific hints and strategies that maximize downstream success of a frozen large answer generator.

Result: Trained concept generator substantially outperforms repeated sampling and untuned large models; transfers to unseen answer generators.

Evidence: Hard mathematical reasoning benchmarks at equal answer-generation compute budget.

The Big Picture

The dominant paradigm for allocating test-time compute to large language models is naive repeated sampling: draw k independent answers from the model at high temperature, then select the best by a verifier or majority vote. This strategy is seductively simple and, at moderate k , effective. But it has a fundamental limitation: each sample explores only through local decoding noise. Temperature and top-p diversify token choices within a trajectory, but they cannot steer the model toward qualitatively different approaches to the same problem.

On hard mathematical reasoning tasks—where correct

solutions often require an insight, decomposition, or analogy that is far from the model’s prior mode—repeated sampling saturates quickly. The k -th sample is nearly as redundant as the second: the model keeps attempting the same dominant strategy with minor surface variations.

This paper asks whether test-time exploration can instead be steered at a *semantic* level. The answer is yes, and the key is to separate the role of *exploration* (finding diverse approaches) from *execution* (implementing a given approach): a small, trainable concept generator proposes strategies; a large, frozen answer generator executes them.

Why Existing Approaches Fall Short

Repeated sampling (best-of- N , majority vote) explores only through decoding stochasticity. Diversity is local; semantically distinct strategies require explicit steering.

Self-consistency aggregates repeated samples but does not improve the diversity of those samples—it is a better aggregator, not a better explorer.

Tree search (MCTS) explores explicitly but requires a well-trained per-step verifier, which is expensive to obtain and domain-specific.

Prompting for diversity (e.g., asking the model to use different methods) provides one alternative per call; emitting a library of diverse concepts in a single trajectory is more efficient.

Scaling the answer generator at inference time is expensive; the goal is to make a fixed frozen model explore more effectively with a lightweight companion.

Core Mechanism

Concept-Conditioned Generation. Given a problem x , the method first samples a *concept* c —a natural-language hint, strategy keyword, or approach description—then generates an answer from the large model conditioned on (x, c) . The concept shifts the answer generator toward a different region of solution space.

Multi-Concept Trajectories. Rather than one concept per call, the concept generator emits a diverse list of concepts in a single forward pass, covering different strategies before any answer is generated. This maximizes spread at minimal additional cost.

Reinforcement-Learned Concept Generator. A small model π_θ is trained with RL to emit concepts that maximize the downstream binary reward (answer correctness). The reward flows back to the concept generator but not to the frozen answer generator f .

Technical Formulation

Let x denote a problem, c a concept sampled from $\pi_\theta(\cdot | x)$, and f the frozen large answer generator. The concept-conditioned pass@k:

$$\text{pass@k}(x) = 1 - \frac{\binom{n-s}{k}}{\binom{n}{k}}$$

where n total samples are generated and s are correct. The RL objective:

$$\max_{\theta} \mathbb{E}_{x \sim \mathcal{D}} \left[\text{pass@k}_{c \sim \pi_\theta(\cdot | x)}(f(x, \cdot)) \right]$$

The policy gradient update uses binary correctness as the scalar reward:

$$\nabla_{\theta} \mathcal{L}(\theta) = -\mathbb{E}_{c \sim \pi_{\theta}} [R(x, c) \nabla_{\theta} \log \pi_{\theta}(c | x)]$$

where $R(x, c) = \mathbf{1}[f(x, c) \text{ is correct}]$ (or a normalized group-relative variant for variance reduction).

Learning or Inference Procedure

Training:

1. Initialize π_θ from a small pretrained LM.
2. For each training problem x : sample K concepts from π_θ , query f for each, record correctness.
3. Update π_θ with policy gradient on the correctness reward.

Inference:

1. Sample a batch of concepts $c_1, \dots, c_K \sim \pi_\theta(\cdot | x)$.
2. Generate answers $a_i = f(x, c_i)$ for each concept.
3. Return the most frequent or highest-confidence answer.

What the Guarantee Says

The trained concept generator is a reusable search policy: it does not learn to solve problems directly but learns which types of strategies lead the answer generator to success on the training distribution. Transfer to new answer generators follows because the learned strategies are problem-structural rather than model-specific.

Experimental Findings

- Hard mathematical reasoning benchmarks where repeated sampling pass@k plateaus at high k
- Concept-conditioned generation with trained π_θ achieves substantially higher pass@k than naive repeated sampling at equal answer-generation compute budget
- Trained small generator outperforms concepts from untuned models many times larger, confirming that RL training—not model scale—is the key factor

- Zero-shot transfer to an answer generator from a different model family succeeds: concepts learned for one model improve another
- Multi-concept trajectories (many concepts per call) further improve pass@k by reducing concept redundancy

Ablations and Interpretation

Single vs. multi-concept: Emitting a diverse list of concepts in one trajectory reduces redundancy and improves pass@k compared to sampling one concept per call.

Generator size: Smaller trained generators outperform larger zero-shot generators, demonstrating the value of training over scale.

Reward signal: Binary correctness alone is sufficient as the RL reward; no dense process reward model or verifier is needed during training.

Transfer: Concept quality transfers across answer generators from different model families, suggesting that the search policy captures problem-structural strategies rather than artifacts of a specific model’s output distribution.

Reference

Ismail Labiad, Matthieu Kowalski, Marc Schoenauer, Rémi Munos, Julia Kempe. **Beyond Repeated Sampling: Learning Search Policies for LLM Reasoning.** arXiv:2609.26704, September 2026.

<https://arxiv.org/abs/2609.26704>